WebDrop

BROWSER-NATIVE NEARBY FILE TRANSFER

WebDrop

In-Depth Guide

VERSION 1.0.87

A first-principles tour of how WebDrop works end to end: the three-lane architecture, WebSocket/WebRTC, NAT/STUN/TURN/ICE, the encrypted data channel and receive ladder, the ultrasonic + motion proximity ceremony, reservation-TDMA scheduling, the proximity scoring model, the concurrent cohort capacity model, the security model, and the interface catalog.

English edition

01 / OVERVIEW

What WebDrop is

WebDrop is a browser-native nearby file-transfer app - AirDrop-style, but with zero install. You open a web page, see nearby devices orbiting your own avatar, pair with one by tapping the phones together (or scanning a QR code), and then files move directly browser-to-browser, encrypted, over WebRTC. The signaling server only coordinates - it never sees a single file byte.

1.0.87	3	256 KiB	500 MB	~50
app version	independent lanes	file chunk size	per-session cap	concurrent pairs

The three rules that define the product

Trust before controls
No transfer UI appears until a verified connection exists. Send / receive / chat are never first-screen affordances.

Metadata on signaling, bytes on WebRTC
The WebSocket lane carries only small JSON (presence, invites, the proximity ceremony, SDP/ICE). File bytes ride an encrypted WebRTC data channel - never the server.

Receiving never surprises you
Incoming bytes are buffered safely and only exported to a download when the user taps Save - a received file can never auto-start a download.

What's inside

- The three-lane architecture and how a transfer runs end to end.
- First-principles explainers: HTTP/WSS, WebRTC, SDP, NAT/STUN/TURN/ICE, DTLS/SCTP.
- The data channel, chunking + backpressure, and the receive storage ladder.
- The proximity ceremony: ultrasonic chirps, bump/tilt, reservation-TDMA, scoring.
- The concurrent-cohort capacity model, the security model, and the UI catalog.

02 / ARCHITECTURE

The three-lane architecture

WebDrop deliberately separates three concerns into independent lanes. Keeping them apart is what makes the system cheap, private, and reliable.

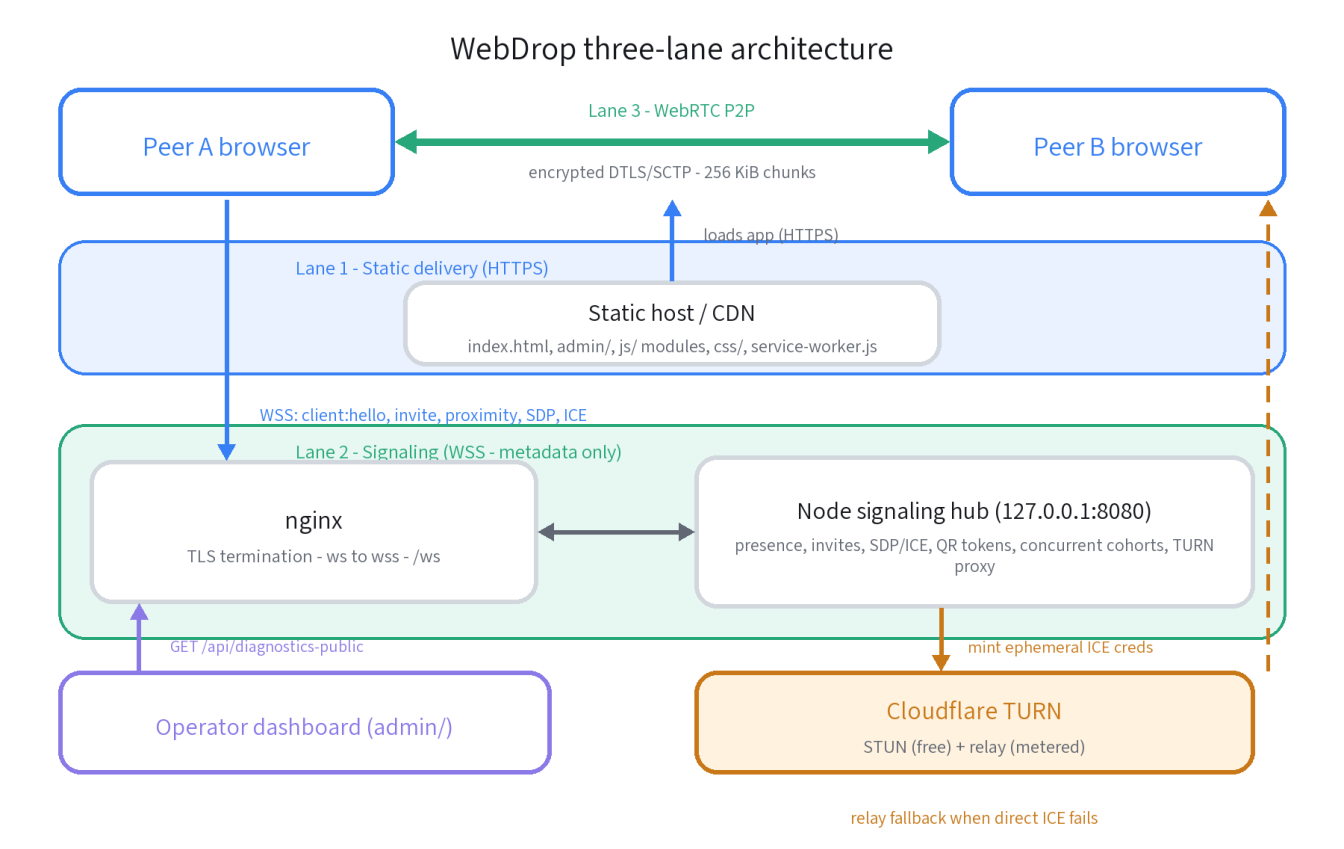


Figure 1 - WebDrop three-lane architecture

- Lane 1 - Static delivery (HTTPS): plain HTML/CSS/JS served over HTTPS. No server-side rendering, no bundler. HTTPS is mandatory because mic, motion, service workers, and WebRTC only work on a secure origin.
- Lane 2 - Signaling (WSS, metadata only): a ws:// WebSocket that nginx upgrades to wss:// (TLS). It carries presence, invites, the proximity ceremony, and the WebRTC handshake (SDP + ICE). No file bytes, ever.
- Lane 3 - Data (WebRTC): an encrypted RTCDataChannel between the two browsers. STUN discovers a direct path; Cloudflare TURN relays only if the direct path fails.

System invariant
The signaling server is a coordinator, not a file server. It must never accept or relay file bytes - only the two browsers ever touch the payload.

03 / FLOW

How a transfer runs, step by step

The same interfaces run locally and in production. Permissions are requested only from the Connect gesture; the transfer dock appears only after a verified connection.

1. Discover. The signaling service announces small presence records, so nearby devices appear on the orbit. No file payload is sent.
2. Choose and verify. The sender taps a candidate and swipes Connect. This is the only place permissions (mic/motion) are requested, from that gesture.
3. Pairing ceremony. Both phones emit and listen for ultrasonic chirps and capture a bump + tilt. The server reveals identities only when the evidence is reciprocal, consistent, and correctly timed.
4. Negotiate WebRTC. The signaling lane relays the SDP offer/answer and ICE candidates; WebRTC then tries a direct path and falls back to TURN relay.
5. Move file chunks. The file is sliced into 256 KiB chunks and streamed over the data channel while watching bufferedAmount for backpressure.
6. Store, then export. Incoming chunks defer into IndexedDB (or a capped Blob on iOS). Only when the user taps Save are they streamed to a download via StreamSaver. Transfer-finished and download-started stay separate.

Foundations: the web platform

WebDrop ships as a tiny static app and relies on the browser's own capabilities. Here is each foundation in plain language, with why WebDrop chose it.

Static HTML / CSS / JS over HTTPS

What it is: three plain text files the browser downloads and runs - HTML (structure), CSS (looks), JavaScript (behaviour) - with nothing to install; HTTPS is just the encrypted version of that download. Like a pop-up book: HTML is the pages, CSS the art, JS the pull-tabs that make things move.

Three text files - structure (HTML), looks (CSS), behaviour (JS) - that the browser downloads and runs, with nothing installed. index.html is a fixed DOM contract; the app flips attributes like data-mode and data-theme on the root element, and CSS reacts. Why: it keeps the app tiny and fast, and HTTPS is mandatory anyway for the powerful sensor APIs.

ES modules

What it is: JavaScript's built-in way to split code across files that import and export pieces from each other - like labelled Lego bricks that each declare what they offer and what they need, snapped together by the browser.

Native import/export with <script type="module">. js/app.js imports the store, view, services, transport, and controller - no webpack/rollup. Imports carry a ?v=1.0.87 cache-buster tied to the app version. Why: a small static app does not need a build pipeline, and native modules give clean dependency boundaries.

Service worker and offline

What it is: a small background script the browser keeps running even after the page closes, able to answer the page's network requests - like an office receptionist who keeps copies of common documents, so you get them instantly even when the post office is shut.

A background script the browser keeps even when the page is closed; it can answer network requests. service-worker.js pre-caches the app shell (CACHE_NAME = webdrop-v2-static-1.0.87) and always fetches runtime-config.js fresh so the live server URL is never stale. Why: the UI loads instantly and works offline while the config that points at the live server stays current.

WebSocket and the WSS/TLS upgrade

What it is: a WebSocket is one long-lived connection both sides can talk over at any moment, instead of asking again and again - like leaving a phone line open rather than mailing postcards back and forth. WSS is just the encrypted (TLS) version: wss:// is to ws:// what https:// is to http://.

A WebSocket is a long-lived, two-way connection over one TCP socket. It is born as an ordinary HTTP request that asks to be upgraded; the server proves it understood and switches protocols. With wss://, TLS wraps it and nginx terminates the certificate, forwarding the upgraded socket to Node on 127.0.0.1:8080.

```
GET /ws HTTP/1.1
Host: webdrop-wss-0618.japaneast.cloudapp.azure.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

After the 101, the first frame the app sends must be client:hello; the server replies connected with a sessionId and an ephemeral turnAccessToken. The hub is strict: binary frames are rejected, ws is created with maxPayload = MAX_JSON_BYTES, every message is schema-validated, and there are per-IP and per-client rate limits. Why: WebSocket is perfect for tiny, frequent coordination messages - but it is not the file lane.

WebRTC, NAT, and finding a path

WebRTC lets two browsers talk directly. WebDrop uses only the data part. Getting two NATed devices to connect is the hard problem that SDP, STUN, TURN, and ICE exist to solve.

WebRTC and the SDP offer/answer

What it is: WebRTC is the browser's built-in way for two devices to talk directly to each other - audio, video, or (here) raw data - with no server in the middle. SDP is the short text 'business card' each side swaps first: here is what I support and how to reach me.

An RTCPeerConnection manages the whole link: describing capabilities, gathering candidates, testing paths, and hosting data channels. SDP is a small text blob - "here's what I support and how to reach me". The caller createOffer() -> setLocalDescription -> sends it; the callee setRemoteDescription -> createAnswer() -> sends back. The server only relays these as rtc:signal. The a=fingerprint line in the SDP later authenticates the encrypted channel.

NAT and why some calls need a relay

What it is: NAT (Network Address Translation) lets many home or office devices share one public internet address, hiding them behind it - like one company phone number with an internal switchboard. Good for privacy, but it means outsiders cannot simply dial a specific device directly.

NAT shares one public IP among many devices and blocks unsolicited inbound traffic, so two NATed peers cannot simply dial each other. Full/restricted-cone NATs reuse one public port and are direct-friendly. Symmetric NAT picks a different public port per destination, which defeats hole-punching - that is the classic reason a call falls back to a TURN relay.

STUN, TURN, and ICE

What it is: STUN is a quick way to ask 'what does my public address look like from outside?' - like asking a friend across the room to read your name tag. TURN is a relay that forwards your data when no direct path works - a post office that re-mails parcels for you. ICE is the process that tries every option and picks the best one that works.

STUN tells you your own public address (a srflx candidate) - cheap and stateless. TURN is a relay that forwards your data when no direct path works - it needs credentials and costs bandwidth, so it is a last resort. ICE ties them together in three phases: gather host/srflx/relay candidates, run connectivity checks (the outgoing probe punches the NAT hole the reply needs), then nominate the highest-priority working pair. WebDrop fetches ICE servers from GET /api/ice-servers; after connecting, classifyPathFromStats() labels the path direct or relay and the UI caps relay transfers at 500 MB.

DTLS / SCTP - the encrypted data channel

What it is: DTLS is TLS - the same padlock encryption as HTTPS - applied to the small packets WebRTC sends, so every byte is scrambled in transit. SCTP rides on top to deliver those packets in order and without loss. Together they are like a sealed, numbered courier pouch.

Once ICE picks a path, the browsers run a DTLS handshake (TLS for datagrams), exchanging certificates and verifying the fingerprint that was pinned in the SDP - so a man-in-the-middle cannot slip in. SCTP then runs over that secure channel to give ordered, reliable, message-framed delivery. So every byte is encrypted automatically. WebDrop still adds manifests and per-file SHA-256, because encryption protects the pipe, not the file boundaries or integrity.

06 / DATA CHANNEL

The data channel, chunks, and backpressure

Once a path exists, file bytes ride an RTCDataChannel - ordered and reliable like TCP. WebDrop keeps control metadata and file bytes on separate channels so neither blocks the other.

Two ordered channels + 256 KiB chunks

What it is: an RTCDataChannel is the WebRTC pipe that carries arbitrary data - not just audio/video - between the two browsers, ordered and reliable like a TCP connection. A 'chunk' is just one bite-sized piece of a big file: sending a book one page at a time rather than all at once.

WebDrop opens two ordered channels (webdrop-control-v1 and webdrop-file-v1). The sender slices each File into 256 KiB chunks (DEFAULT_CHUNK_SIZE), sends a manifest first (file ids, sizes, chunk counts, per-file SHA-256), then streams chunks while watching RTCDataChannel.bufferedAmount. Each session is capped at 500 MB (DEFAULT_SESSION_CAP_BYTES). 256 KiB is small enough to keep retry bookkeeping cheap on mobile, large enough to avoid per-message overhead.

Backpressure, worked

What it is: backpressure means slowing the sender when the receiver or network buffer is filling up, so nothing overflows - like pausing while pouring water into a cup so it never spills over the rim.

A 250 MB file at 256 KiB/chunk is ~1,000 chunks. Calling send() 1,000 times in a loop would queue all 250 MB in the browser send buffer and can crash the tab. Instead the sender pauses at a high-water mark and resumes on bufferedamountlow, so only a few chunks are ever in flight - RAM stays bounded regardless of file size.

```
if (channel.bufferedAmount > HIGH_WATER) {
  await once(channel, "bufferedamountlow"); // let the network drain
}
channel.send(chunk);
```

The receive ladder

What it is: IndexedDB is a small database built into the browser for storing data on the device; StreamSaver streams incoming bytes straight into the browser's normal download. Together they mean a huge file never has to sit in memory all at once.

The receiver must avoid assembling a multi-hundred-MB file in RAM, and a web page cannot silently write to disk. storage-client.js climbs a ladder, in order of preference:

1. IndexedDB (capable non-iOS browsers): each ordered chunk is written while receiving, keyed by session/file/chunk. No download starts yet.
2. On Save, stored chunks stream through the self-hosted StreamSaver helper into the browser's download pipeline.
3. iPhone/iPad Safari: a capped Blob fallback assembles chunks in memory and offers Open (preview) - capped because memory grows with the payload.
4. Compatibility fallback: direct receive-time StreamSaver writing when IndexedDB is unavailable; if no safe backend can hold the size, the file is rejected before any byte is accepted.

Transfer finished is not download started
Incoming chunks defer into storage and only export to a download when the user taps Save. A received file can never auto-start a download.

07 / SENSING

Proximity sensing: sound and motion

The pairing ceremony gathers physical evidence that two phones are really next to each other: a near-ultrasonic chirp the mic can hear, and a bump plus tilt the accelerometer can feel.

Ultrasonic chirps (Web Audio)

What it is: the Web Audio API lets a web page generate and analyse sound through the speaker and microphone. A 'chirp' here is a short tone that sweeps a near-ultrasonic band - high enough that people barely hear it, but a nearby phone's mic can. Like two crickets recognising each other's call.

acoustic-proximity.js emits a coded chirp in a near-ultrasonic band (DEFAULT_CHIRP: 18,600-19,400 Hz) and records the mic with echo cancellation, noise suppression, and auto-gain control turned off, so the browser does not erase the high-frequency energy. Sound is short-range and direction-ish, so hearing a peer's coded chirp is good evidence the two devices are physically near.

Bump and tilt (DeviceMotion)

What it is: DeviceMotion is the browser API that reads a phone's accelerometer - the same sensor that knows which way is down and feels a shake. WebDrop uses it to feel two phones physically tapped together and tilted, a gesture that is deliberate and hard to fake.

motion-proximity.js detects a bump when linear acceleration >= BUMP_ACCELERATION_THRESHOLD (10) or the gravity-vector change >= 3.5, and a tilt when |beta| or |gamma| > 30 degrees. These feed the proximity score and are also mandatory server-side evidence. Bumping the phones together is a deliberate gesture that is hard to trigger by accident.

Detection, not loudness

What it is: correlation is a math check that asks 'how closely does this recording match the exact pattern we sent?' (1.0 = perfect), rather than 'how loud was it?' - like recognising one specific melody in a noisy room, not just hearing noise.

Detection is not "was it loud?" but "did the recording contain this specific coded sweep, clearly above the background?". Correlation slides the expected template over the recording (1.0 = perfect); the energy margin compares chirp-band energy against nearby background in dB. WebDrop accepts on a ladder (e.g. 0.30 / 0.20 / 0.16 correlation, backed by 4.5 dB / 8 dB margins). The server also keeps the shared band under ~sampleRate x 0.45 - 100 Hz with >= 420 Hz of bandwidth, so it never asks a phone to detect a tone above its Nyquist limit.

> **QR is the dependable fallback**
> When mic/motion are denied or unsupported, one phone shows a one-time, server-issued QR token and the other scans it - camera and a screen are all it needs.

Reservation TDMA: one window, pre-assigned slots

If several nearby phones chirp at once, their sounds collide and nobody decodes cleanly - the classic multiple-access problem. Because the server already knows the cohort, it hands out time slots before anyone transmits.

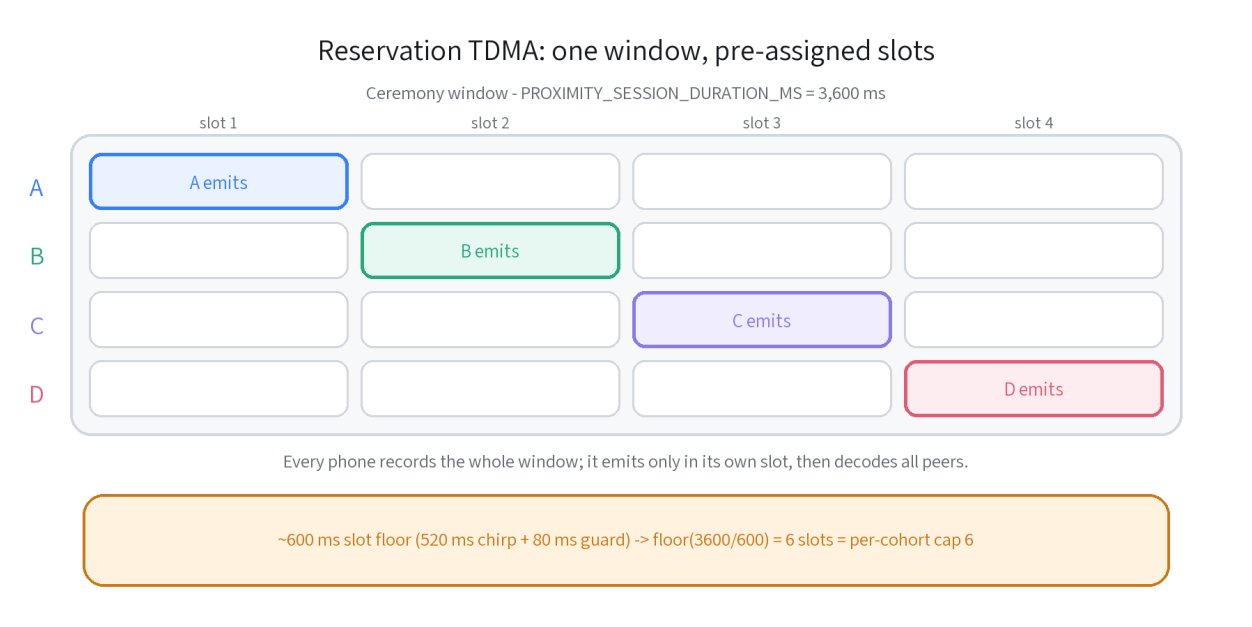


Figure 2 - Reservation TDMA schedule

What it is: TDMA (Time Division Multiple Access) means several senders share one channel by each taking a turn in its own time slot - like people in a meeting agreeing to speak one at a time instead of all at once.

This is reservation TDMA (Time Division Multiple Access): every phone records the whole 3,600 ms window but emits only in its own slot, then decodes all peers afterward. It is Aloha-family - Slotted Aloha lets devices gamble on a slot and back off on collision; WebDrop instead reserves slots centrally, which is strictly more reliable for a small, known, server-coordinated cohort.

- Slot floor: a coded chirp needs ~520 ms + an ~80 ms guard, so ~600 ms per slot.
- floor(3,600 / 600) = 6 slots fit, which is exactly why the per-cohort cap is 6.
- Bigger cohorts need a longer window (PROXIMITY_SESSION_DURATION_MS), not just a bigger number - the cap is clamped to the slot-floor ceiling.

Scoring: reciprocal signatures and the winner margin

Pairing identity is decided by who heard whose coded chirp - not by loudness or "who is nearby". This is what keeps A<->B and C<->D from cross-pairing.

Reciprocal coded signatures (primary)

For A<->B, A must report its own assigned signature and report hearing B's, and B must do the same for A - each above a detection threshold (hasReciprocalAcousticEvidence). A+C never match because A heard B's signature, not C's. A secret handshake only counts if both people do their half and recognise the other's half.

The winner margin (fail-safe)

The winner margin requires the best-matching signature to be sufficiently clearer than the runner-up, so a third nearby phone cannot be mistaken for the partner. The guard fails safe: a missing or non-finite acousticConfidenceMargin fails the check rather than passing by default (ACOUSTIC_WINNER_MARGIN = 0.04).

What verification requires

Requirement	Rule
Score	>= 55% proximity score
Evidence	ultrasound + bump + tilt all mandatory
Reciprocity	both heard each other's signature
Timing	bump times within 4,000 ms; valid ceremony
Winner	unambiguous winner above the margin

Tilt is presence-only: it must be present, but it does not decide which device pairs with which. Exact bump timing is a gate plus a tie-break, never the identity matcher.

10 / CAPACITY

The concurrent-cohort capacity model

Instead of one global pairing session, the hub runs many concurrent bounded cohorts: many small rooms of six at once, under a global headcount cap.

A new joiner slots into an open cohort with room, or opens a fresh one; when a cohort fills (6) it closes and starts its ceremony. MAX_TOTAL_PROXIMITY_PARTICIPANTS (default 100) bounds everyone - beyond it, joins fail cleanly with reason: capacity_reached before any cohort is mutated. So 100 / 6 = ~17 cohorts = up to ~50 pairs. The per-cohort cap MAX_PROXIMITY_SESSION_CLIENTS (default 6) is clamped to the slot-floor ceiling, so the scheduler can never create sub-floor acoustic slots.

Tuning knobs (and what each one trades)

Knob	Default	Raising it...
MAX_TOTAL_PROXIMITY_PARTICIPANTS	100	more concurrent participants; needs Redis + multi-node to mean anything
MAX_PROXIMITY_SESSION_CLIENTS	6	no effect unless the window grows (clamped to the slot floor)
PROXIMITY_SESSION_DURATION_MS	3,600 ms	adds slots so bigger cohorts become legal; slower ceremony
ACOUSTIC_SESSION_STAGGER_MS	600 ms	spreads simultaneous cohort starts so they don't all chirp at once
ACOUSTIC_MAX_CONCURRENT_SUBBANDS	4	splits cohorts across frequency lanes; a no-op until the band widens

Honest caveat
The software allows ~50 co-located pairs, but ~50 pairs sharing one ~800 Hz band in one room is acoustically contended - real reliability is a physical-device question. The path to 10,000 is config + Redis/shared presence + sticky multi-node WS + staged load testing.

11 / Q&A

Multi-device pairing, answered

The questions testers ask most about pairing many phones at once - and what the current code actually does.

A+B and C+D - how is A+C avoided?

The primary disambiguator is reciprocal signatures, not who is nearby. A heard B's coded chirp and B heard A's, so A+C never match. Bump-time (within 4,000 ms) is a gate and tie-break only.

What if one device taps Connect later?

A cohort's join window is 1,800 ms with one 1,800 ms extension (~3.6 s total). Tap within it and you join the same cohort; tap after it starts and you land in a different concurrent cohort. A device left alone fails with no_nearby_partner.

Do groups in different rooms interfere?

No. Logical: pairing is gated by reciprocal coded signatures, so a stray chirp carries the wrong signature and fails the match. Physical: ultrasound is directional and attenuates fast, and cohorts are start-staggered and sub-banded.

What is the maximum, and what shows at the cap?

The hard ceiling today is 100 simultaneous in-ceremony participants (~50 pairs), a single-node in-memory bound. A device tapping Connect at the cap gets a clean capacity_reached and can retry or use QR - nothing pairs incorrectly.

12 / STATE MACHINE

The UI state machine and the control gate

The interface is a small explicit state machine. The single most important rule is that the transfer dock only exists after a verified connection.

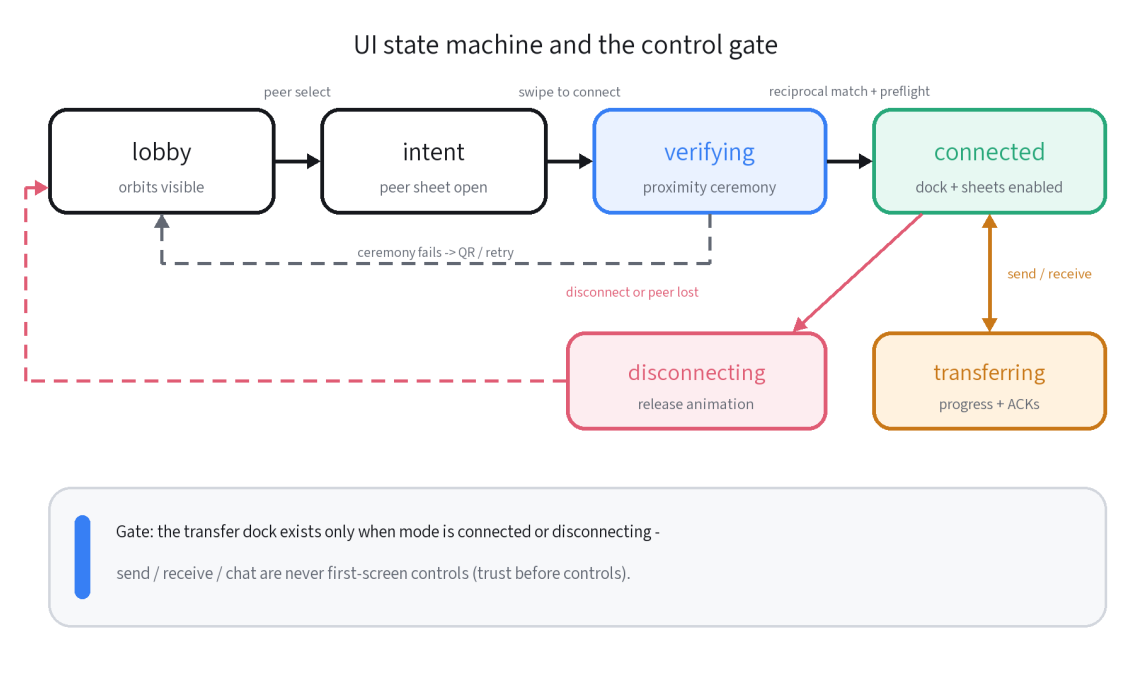


Figure 3 - UI state machine and the control gate

The runtime drives the root element's data-mode through lobby -> intent -> verifying -> connected -> disconnecting; transfer is tracked as a progress object rather than its own mode. The gate in AppView.renderTray() is the enforced version of "trust before controls":

```
const connected = state.mode === "connected" || state.mode === "disconnecting";
this.nodes.tray.hidden = !connected;  // the dock exists only after a verified link
```

Trust before controls
Send / receive / chat are never first-screen affordances - the dock appears only once a reciprocal, proximity-verified connection exists.

The security model

The client trusts only direct user gestures in the current tab and the file handles the user picked. It distrusts peer-declared names, MIME types, and any message/SDP/ICE outside the expected pairing session or schema.

Frontend hardening

- XSS-safe previews: peer-declared dangerous MIME (text/html, image/svg+xml, any text/*, XML) are download-only and never opened in-page. Only images, video, and PDF are previewable.
- No object-URL leaks: received-file object URLs are revoked after use, and file names are escaped before rendering.

Backend hardening

- Enforces allowed origins, rejects binary frames, caps payloads (ws maxPayload = MAX_JSON_BYTES), and schema-validates every message.
- Rate-limits per IP and per client, mints only ephemeral TURN credentials, and redacts secrets in logs (authorization, token, credential, iceServers, sdp).
- With proximity analysis live, the server blocks RTC/chat/path/transfer routing until both peers reach a verified decision (score >= 55% + mandatory evidence).

Diagnostics access

The operator dashboard reads one authenticated endpoint, GET /api/diagnostics-public, which requires the METRICS_API_TOKEN bearer. The payload is metadata-only - never TURN credentials, QR tokens, raw microphone audio, or file bytes.

14 / OPERATIONS

The operations dashboard

admin/index.html (driven by readiness.js via diagnostics-api.js) gives operators a read-only view of fleet health and live pairing telemetry. It never opens its own microphone.

Readiness tab

Environment and health summary from /readyz - which capabilities and credentials the live deployment actually has.

Live testing tab

Bounded device/pair/cohort telemetry from /api/diagnostics-public, plus single-device acoustic diagnostics via admin:monitor:*.

admin/diagnostics.html is now just a redirect to admin/index.html?tab=live. The dashboard surfaces only metadata, so an operator can watch readiness and live cohorts without ever touching a file byte or a credential.

Limits and honest caveats

WebDrop is deliberately scoped. These are the current, real limits - stated plainly so expectations match the code.

- Session cap: each send and receive session is capped at 500 MB (DEFAULT_SESSION_CAP_BYTES); relay (TURN) paths are held to the same ceiling.
- Concurrency: ~100 simultaneous in-ceremony participants (~50 pairs) on a single in-memory node; reaching 10,000 needs Redis + multi-node sticky WS + load testing.
- Acoustic reliability is physical-device dependent: band, gain, slot length, and orientation effects must be re-measured on real phones after any acoustic change.
- Platform limits: HTTPS/secure origin is mandatory for mic/motion/WebRTC; iOS Safari uses a capped in-memory Blob for large receives instead of IndexedDB.

> **At the cap, nothing breaks**
> A device that taps Connect when the ceiling is full gets a clean capacity_reached before any cohort is mutated; it can retry a moment later or fall back to QR. 100 is a safe default, not a product limit.

WHAT'S NEXT

Production activation

The static client and the signaling backend are built and hardened. The remaining work is operational: calibrate the proximity ceremony on real devices, prove direct and relay transfers at scale, and grow capacity toward the 10,000-user target.

Device calibration

Measure ultrasonic detection, bump, and tilt on real iOS/Android phones; tune band, gain, slot length, and thresholds. Acoustic reliability is physical-device dependent.

Transfer proof

Prove direct and TURN-relayed WebRTC transfers between two browsers, including large receives, cancel/retry, and storage exhaustion.

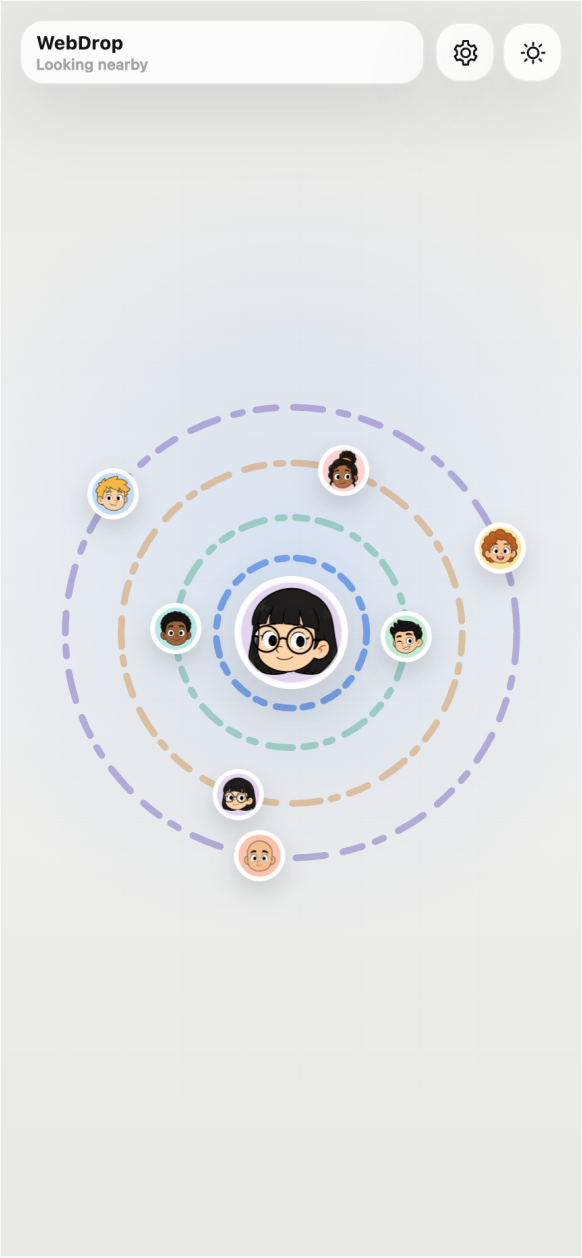
Scale to 10,000

Add shared presence/state (Redis), sticky multi-node WebSocket balancing, staged load testing, and monitoring before raising the global participant cap.

APPENDIX / UI CATALOG 01

Interface catalog

Approved WebDrop 1.0.87 interface inventory

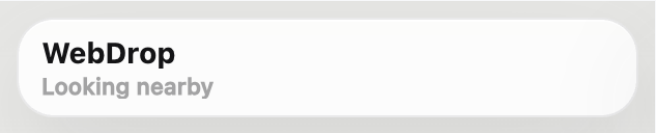


Light home screen

The main nearby-device surface in light mode, with the WebDrop status, four orbits, a gently animated self icon, and static orbit peers.

Interface catalog

Approved WebDrop 1.0.87 interface inventory



02

WebDrop status pill

Shows the WebDrop label and nearby or connected status without exposing transfer controls too early.

03

Settings icon

Opens the settings sheet for profile icon, ring color, language, app information, and version.

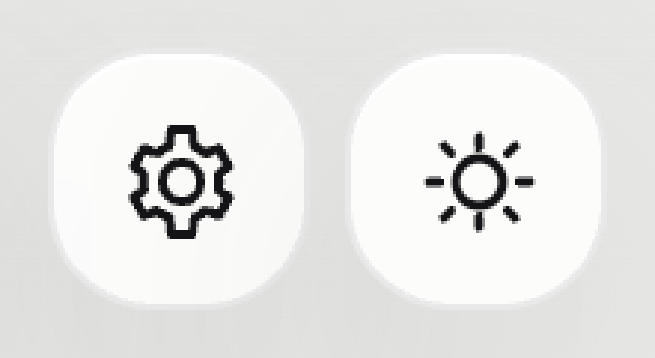
Interface catalog

Approved WebDrop 1.0.87 interface inventory

04

Theme toggle

Switches between light and dark mode from the main screen.



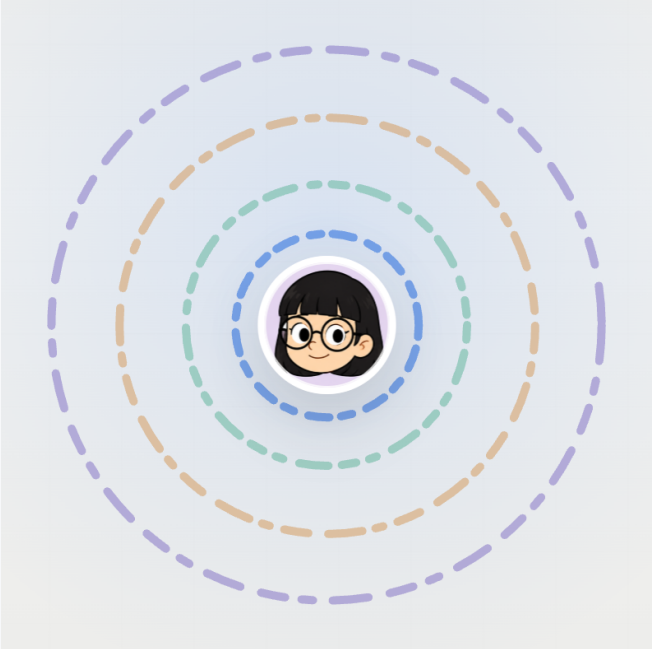
05

Header actions

Settings and theme controls are grouped in the top-right header area.

Interface catalog

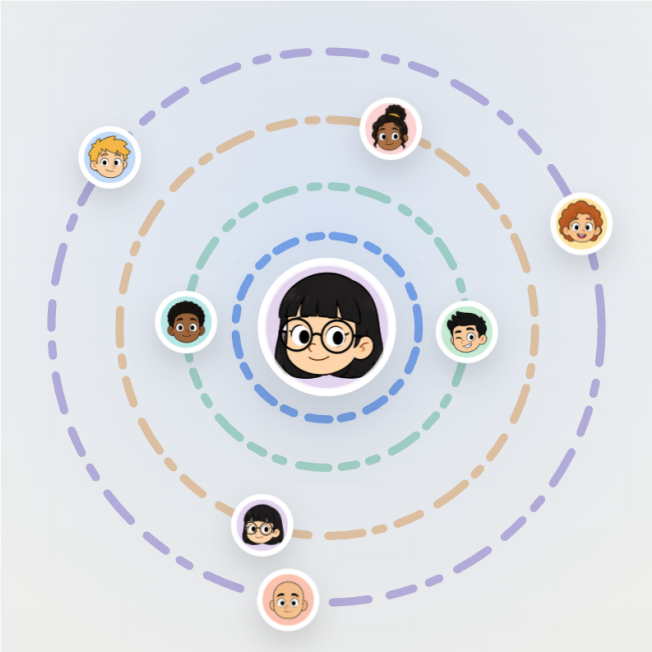
Approved WebDrop 1.0.87 interface inventory



06

Orbit layers without peers

The clean App Clip-inspired orbit pattern before showing nearby devices.



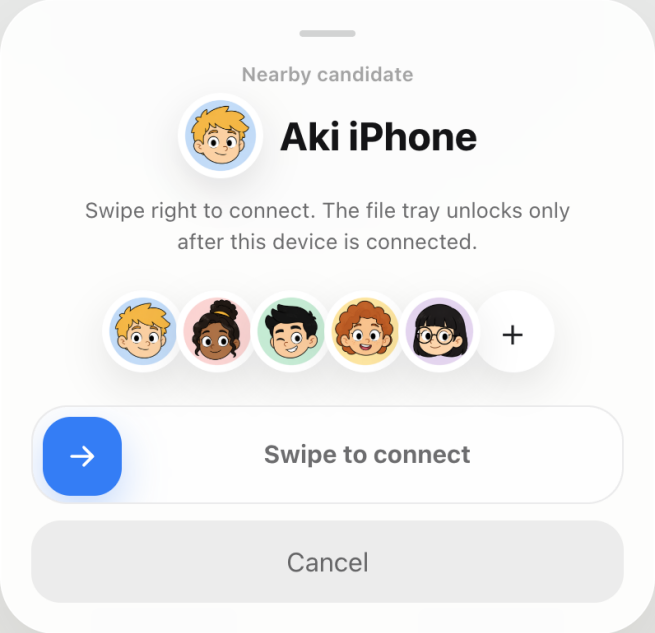
07

Orbit layers with peers

Nearby candidates sit on separate rings with static profile icons so orbital movement remains calm.

Interface catalog

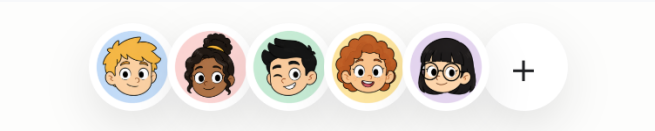
Approved WebDrop 1.0.87 interface inventory



08

Connect sheet

Peer selection opens a bottom sheet where the user confirms by swiping right.



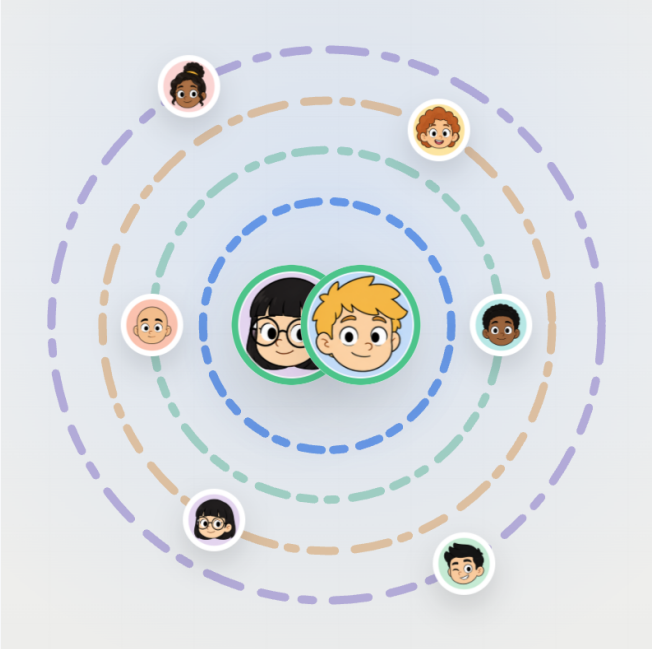
09

Friend strip

The candidate strip uses the same profile style as the orbit icons, with a small overlap and visible circular edges.

Interface catalog

Approved WebDrop 1.0.87 interface inventory



10

Connected Venn state

The current device and selected peer overlap as an equal-size, static Venn-style pair with animated connected rings.

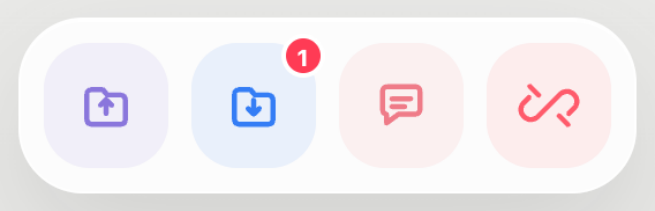
11

Connected status pill

After connection, the status line names the connected peer.

Interface catalog

Approved WebDrop 1.0.87 interface inventory



12

Connected dock

Send, receive, chat, and disconnect appear only after the connection is verified.

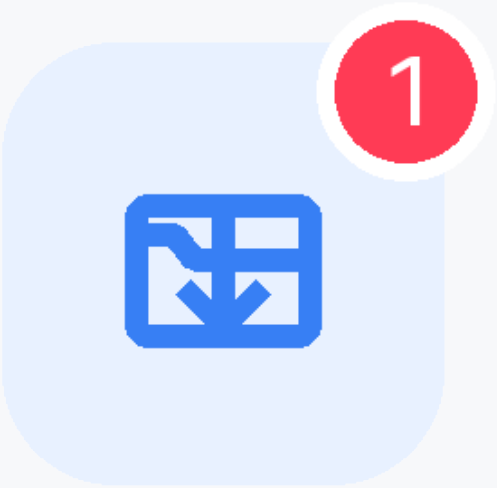
13

Send icon

Opens the send sheet.

Interface catalog

Approved WebDrop 1.0.87 interface inventory



14

Receive icon

Opens received files and shows a badge when files are available.

15

Chat icon

Opens the peer chat sheet.

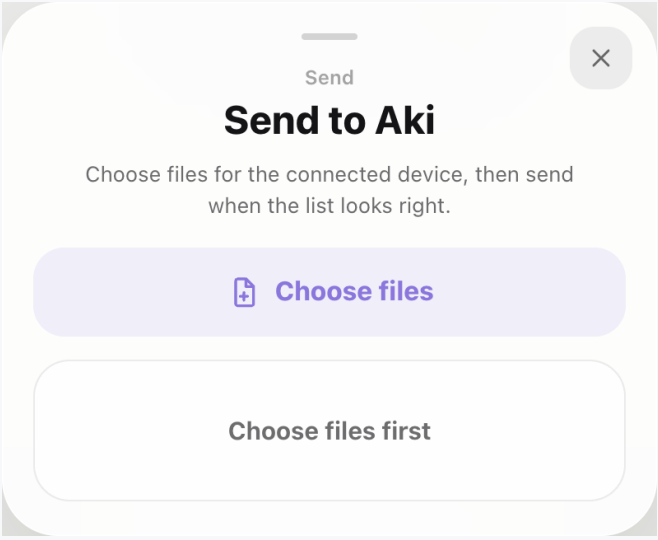
Interface catalog

Approved WebDrop 1.0.87 interface inventory

16

Disconnect icon

Disconnects directly with a short release animation.



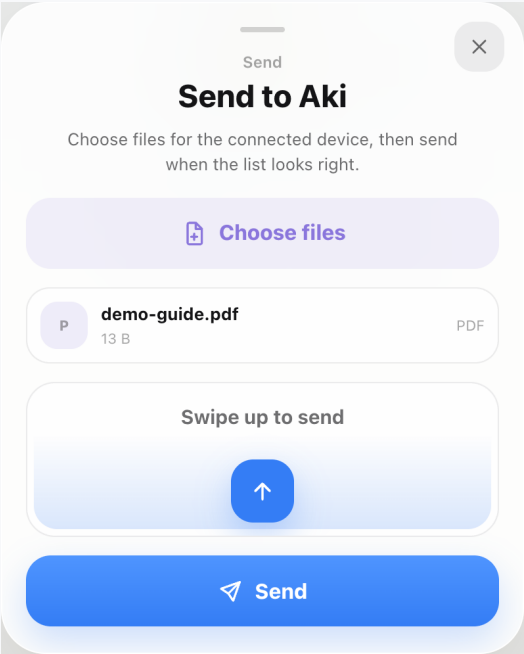
17

Send sheet before file selection

The user chooses files first; the send swipe remains disabled until a file is selected.

Interface catalog

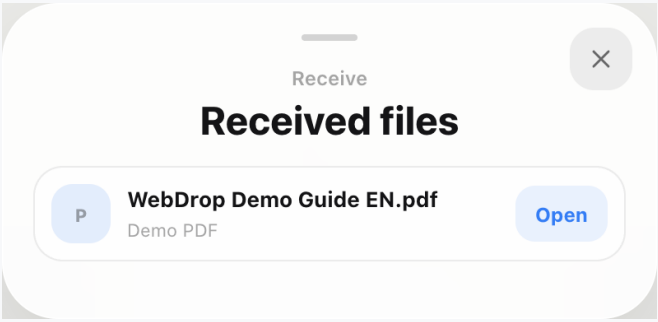
Approved WebDrop 1.0.87 interface inventory



18

Send sheet after file selection

A selected file appears in the list and the swipe-up send control keeps its icon below the text.



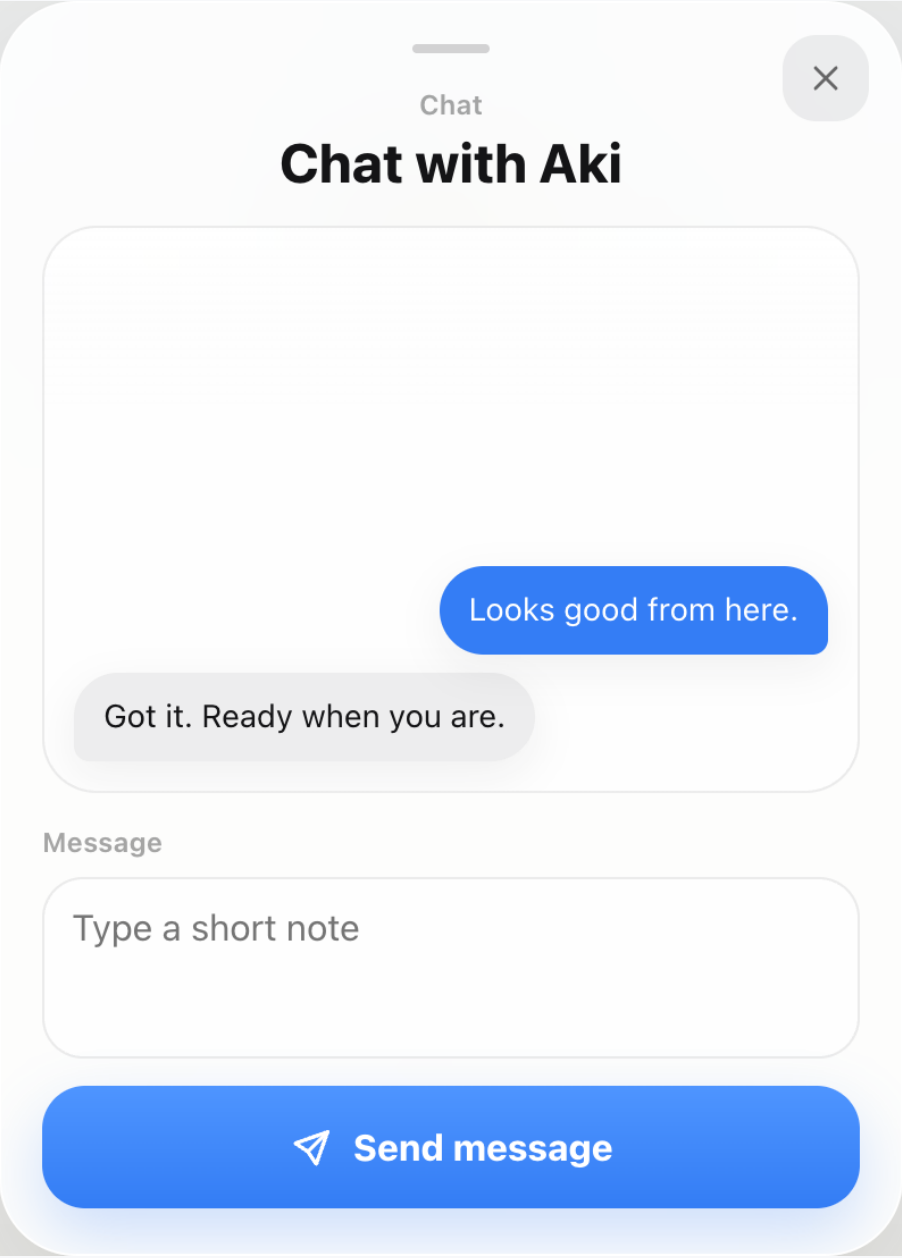
19

Receive sheet

Received demo PDFs appear here with an Open action and the dock badge reflects the count.

Interface catalog

Approved WebDrop 1.0.87 interface inventory

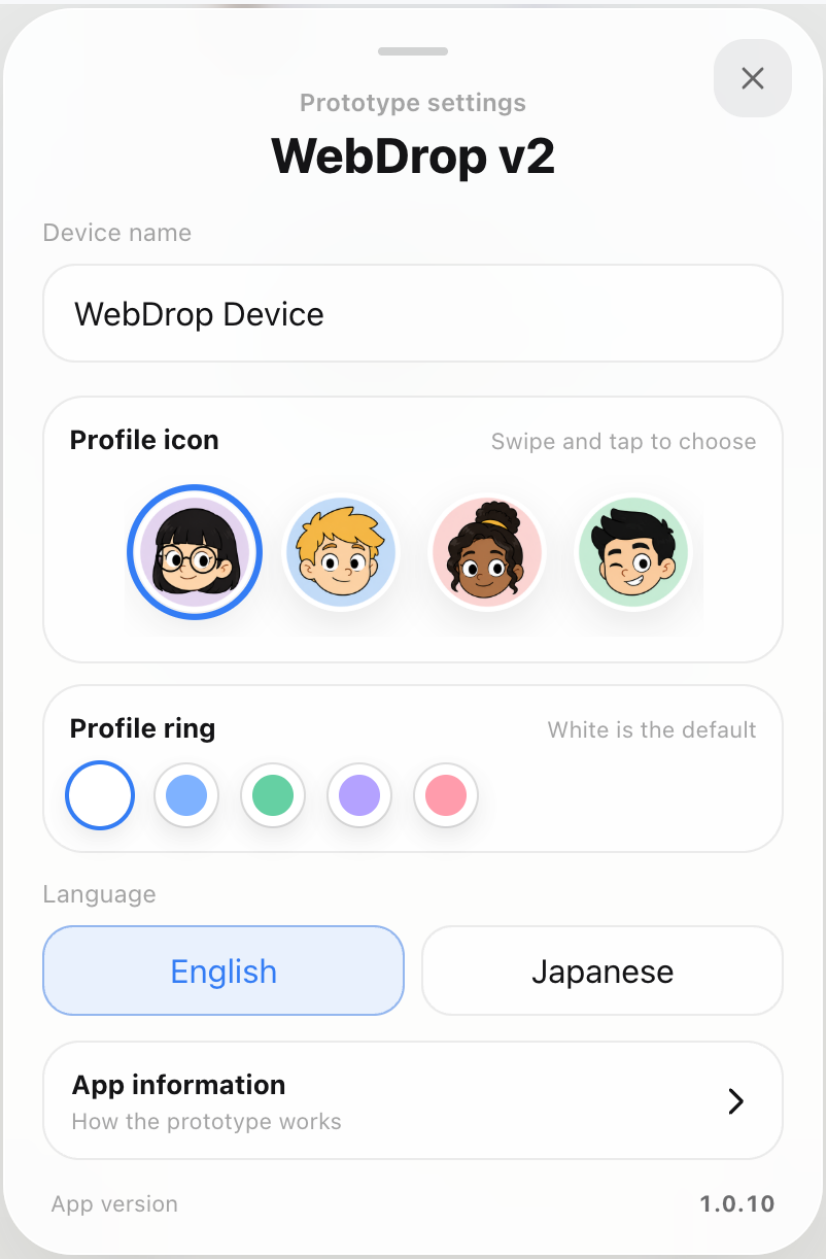


Chat sheet

Chat is separated into its own scrollable bubble conversation instead of sharing the file tray.

Interface catalog

Approved WebDrop 1.0.87 interface inventory

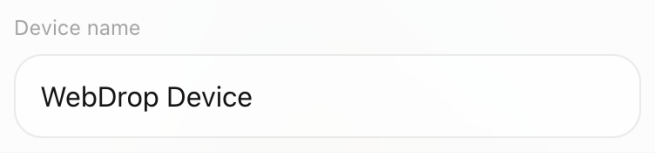


Settings sheet

Settings hold device name, profile, ring, language, app information, and version.

Interface catalog

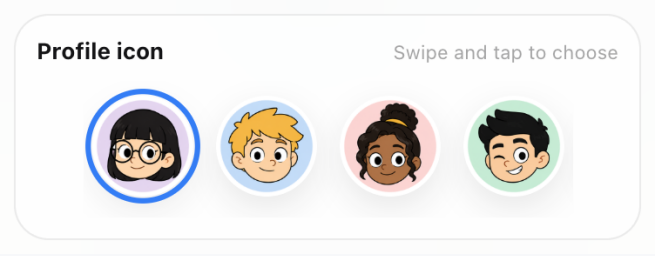
Approved WebDrop 1.0.87 interface inventory



22

Device name field

The device name can be edited, including clearing the final character without an unwanted reset.



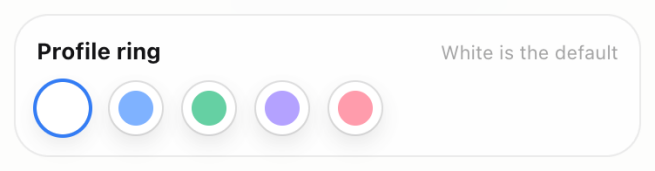
23

Profile icon selector

Users swipe across the provided character icons instead of uploading a photo.

Interface catalog

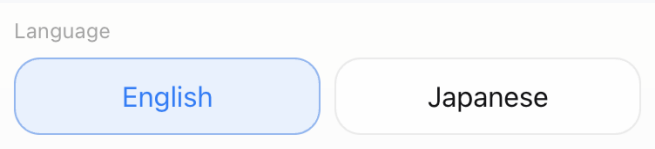
Approved WebDrop 1.0.87 interface inventory



24

Profile ring selector

The default ring is white, with optional blue, green, purple, and rose choices.



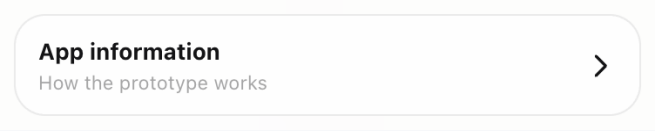
25

Language selector

Switches every app label between English and Japanese.

Interface catalog

Approved WebDrop 1.0.87 interface inventory



26

App information link

Moves design, stack, and orbit motion details into a separate sheet.

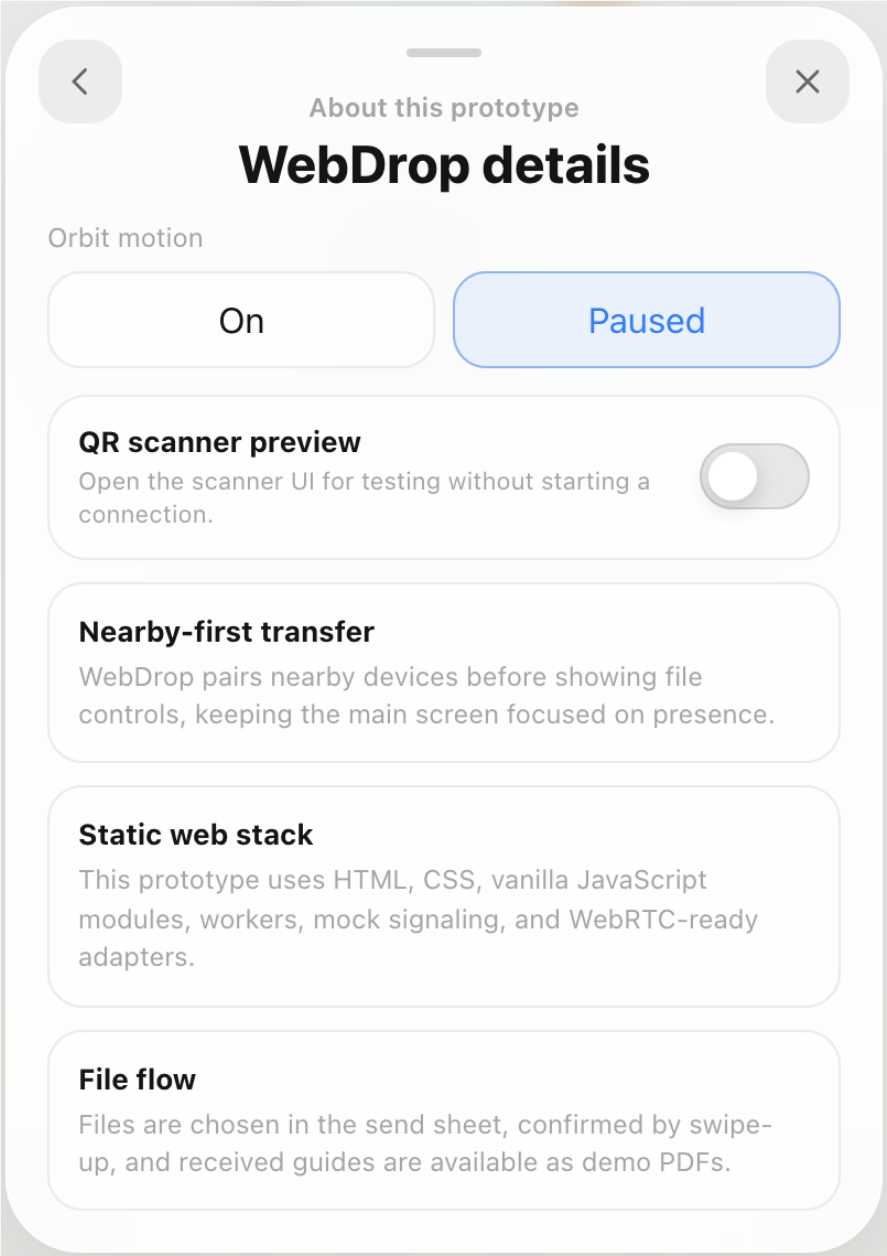
27

App version

Shows the current prototype version, 1.0.87, at the bottom of Settings.

Interface catalog

Approved WebDrop 1.0.87 interface inventory

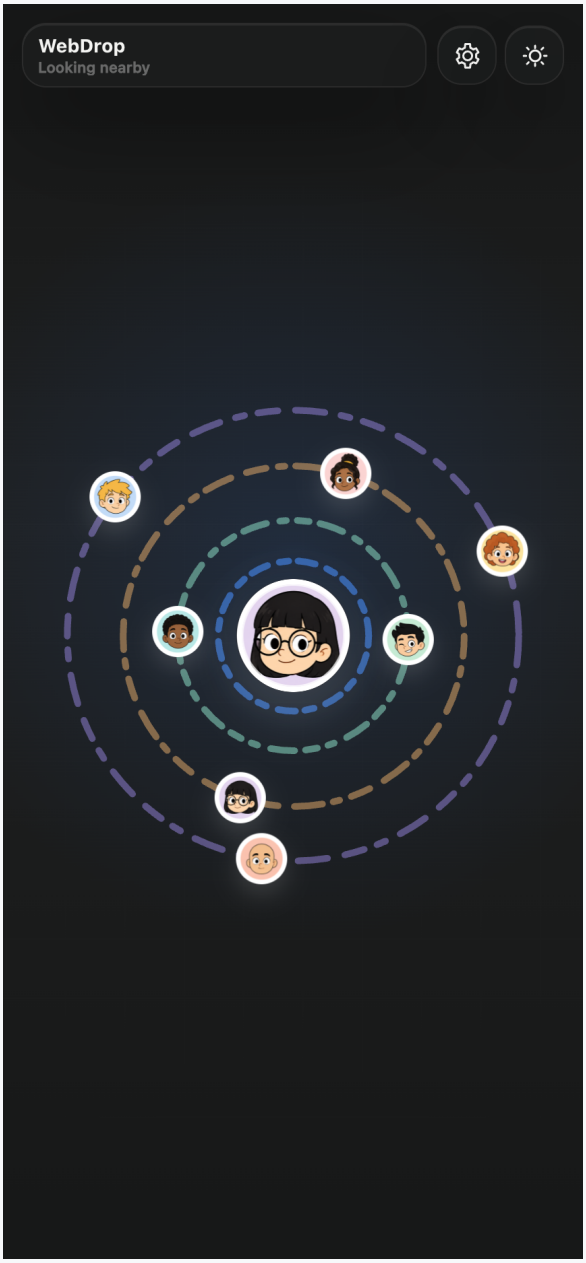


App information sheet

Explains the prototype and contains orbit motion controls without cluttering the settings page.

Interface catalog

Approved WebDrop 1.0.87 interface inventory



Dark home screen

The same nearby-device UI in dark mode.